MBLP131 · HAFTA 7

İşletim Sistemleri Temelleri

Hafta 7 · Sanal Bellek ve Dosya Sistemleri

4 GB'lık bir program 2 GB RAM'de nasıl çalışır ve dosyalar gerçekte nerede durur?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 6. Hafta

İki tür adres

Programlar mantıksal adres kullanır; MMU bunları çalışma anında fiziksel adrese çevirir.

Bitişik yer ayırma

Her sürece tek bir RAM bloğu vermek basittir ama delikler bırakır: dış parçalanma.

Sayfalama

Sayfalar (mantıksal) bir sayfa tablosuyla çerçevelere (fiziksel) eşlenir: dış parçalanma olmaz.

TLB

Sayfa tablosu girdilerinin küçük bir önbelleği. Etkin erişim süresi isabet oranı α'ya bağlıdır.

Geçen hafta bir sürecin her sayfası RAM'deydi. Bugün bu varsayımı bırakıyoruz: bir sayfa diskte de olabilir.

Bugünün Gündemi

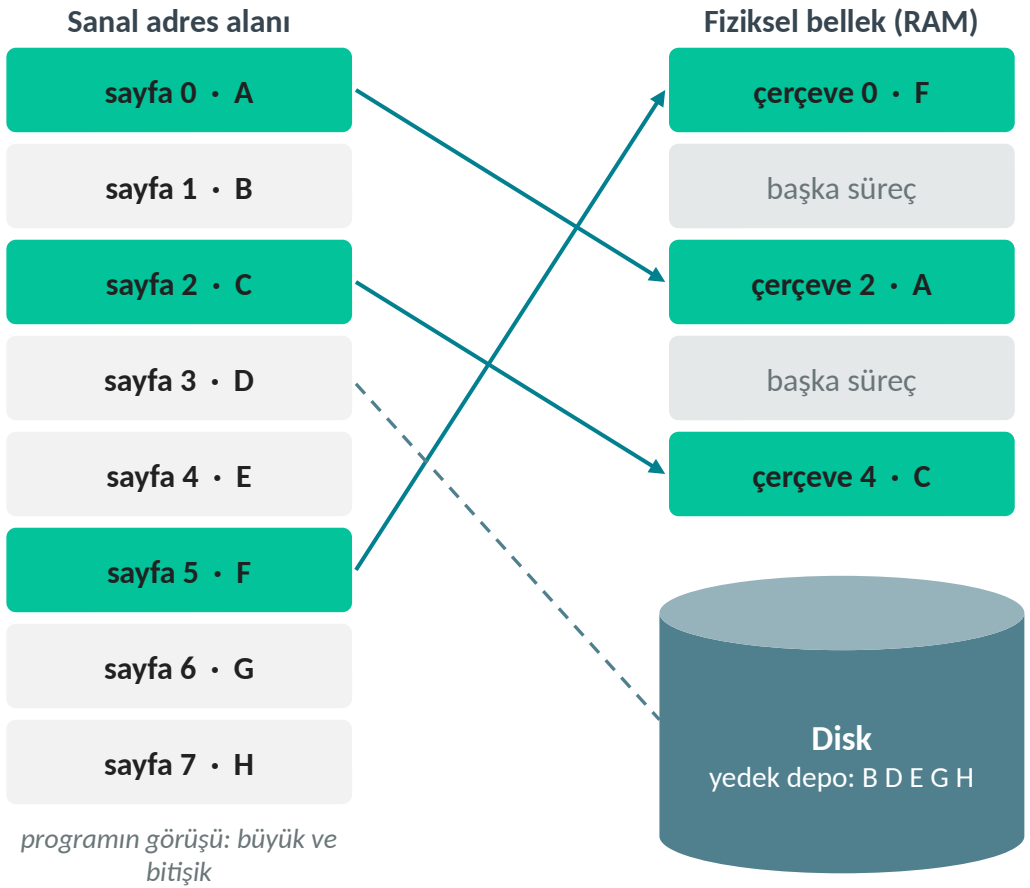
1. **Sanal bellek** Programın yalnızca bir kısmının RAM'de olması yeter: isteğe bağlı sayfalama, sayfa hataları.
2. **Sayfa değiştirme** Bellek dolunca kim çıkar? FIFO, OPT, LRU, Belady anomalisi, saat algoritması.
3. **Boşa çalışma** Çok fazla süreç, çok az çerçeve ve çalışma kümesi fikri.
4. **Dosya sistemleri** Dosyalar, öznitelikler, işlemler, erişim yöntemleri.
5. **Dizinler ve Linux** Dizin yapıları, yollar, düğümler (inode), sabit ve sembolik bağlantılar.
6. **Vize kontrol listesi** Gelecek haftaki sınav için neleri tekrar etmeli (1–7. haftalar).

BÖLÜM 1

Sanal Bellek

Bir programın çalışması için bütün sayfalarının RAM'de olması gerekmez.

Fikir: Programın Görüşünü RAM'den Ayırmak



programın görüşü: büyük ve bitişik

Neden işe yarar?

- Bir programın büyük kısmı nadiren kullanılır: hata işleyicileri, sıra dışı menü seçenekleri, gerekenden büyük tanımlanmış diziler.
- **Yerellik (locality):** kısa bir süre içinde program sayfalarının yalnızca birkaçını kullanır.
- **Öyleyse:** yalnızca kullanılan sayfaları RAM'de tut; gerisi diskte beklesin.
- **Kazançlar:** RAM'den büyük programlar çalışabilir; belleğe daha çok süreç sığar (daha yüksek işlemci kullanımı); daha az şey yüklendiği için programlar daha hızlı başlar.

Hepsini mi Yükleyelim, Yalnızca Gerekeni mi?

İsteğe bağlı sayfalama (demand paging) = sayfalama + tembel yükleme: bir sayfa ancak ilk kullanıldığında RAM'e getirilir.

Hevesli: programın tamamını yükle

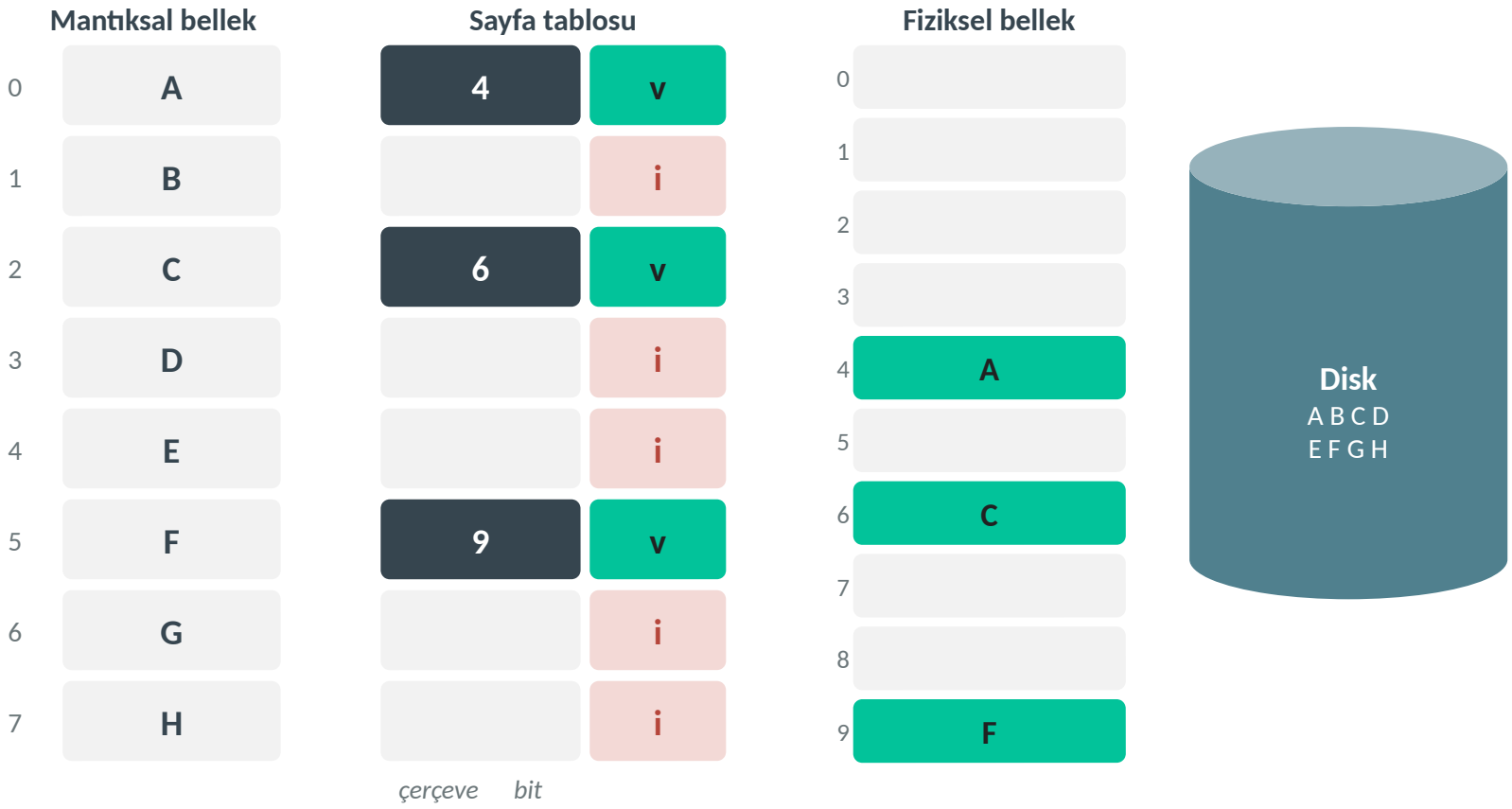
- Program başlamadan önce tüm sayfalar diskten okunur.
- Kullanıcı tek bir menü açacak olsa bile başlangıç yavaştır.
- Hiç kullanılmayan sayfalar için RAM harcanır.
- Bellekte aynı anda daha az program yer alır.

Tembel: isteğe bağlı sayfalama

- Az sayıda (ya da hiç) sayfayla başla.
- Bir sayfayı ona ilk başvurulduğunda yükle.
- Daha az G/Ç, daha az bellek, hızlı başlangıç, daha çok süreç.
- Bedeli: her sayfanın ilk kullanımı bir sayfa hatasına yol açar.

Sayfaları disk ile RAM arasında taşıyan işletim sistemi parçasına sayfalayıcı (pager) denir. "Saf" isteğe bağlı sayfalamada program bellekte hiç sayfa olmadan başlar.

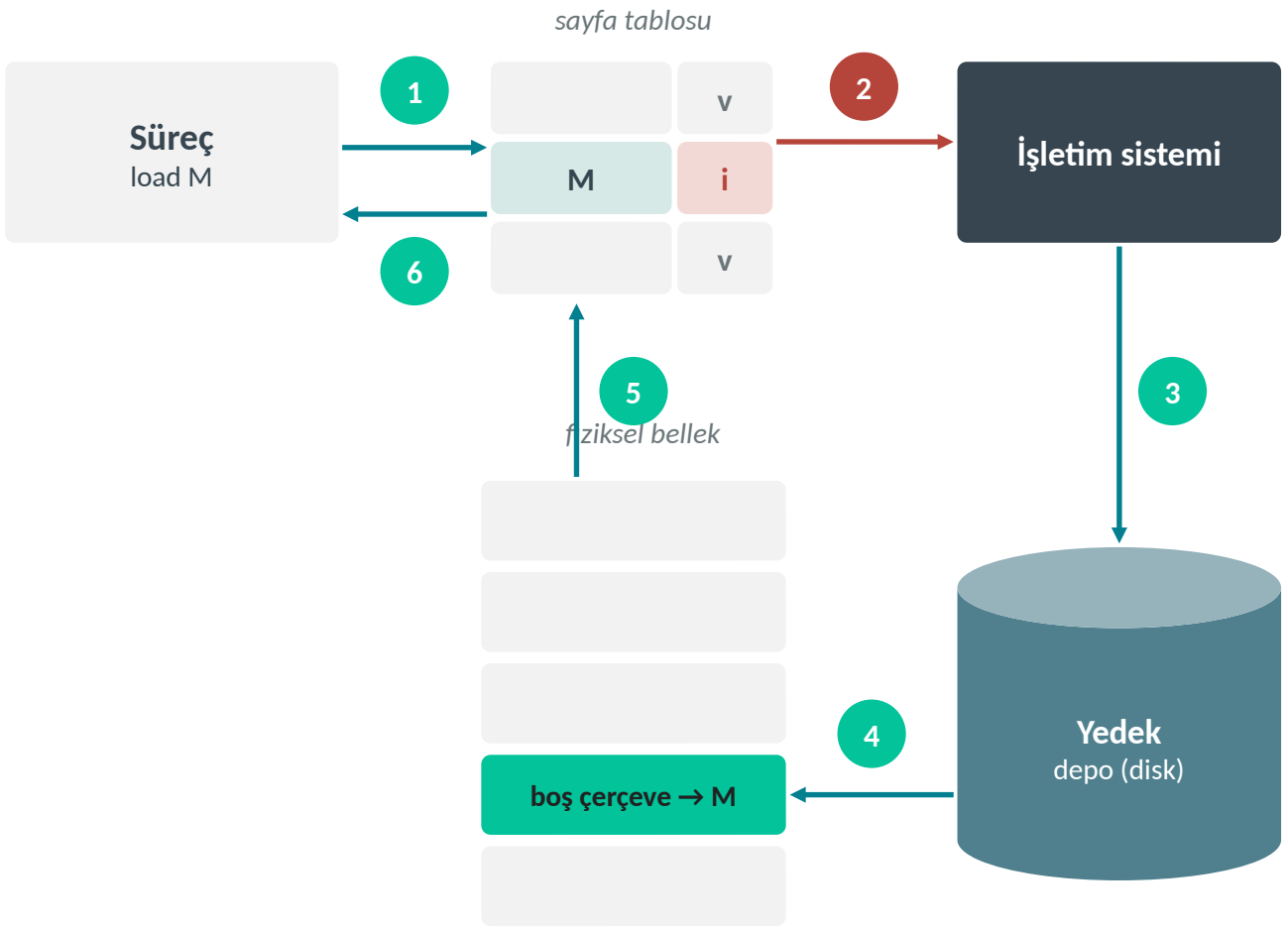
Geçerli–Geçersiz Biti: Sayfa Bellekte mi?



Biti okumak

- **v (valid, geçerli):** sayfa yasaldır ve bir çerçevededir.
- **i (invalid, geçersiz):** sayfa bellekte değildir (ya da programın parçası bile değildir).
- Başlangıçta her girdi i'dir.
- i olan bir sayfaya dokunulunca MMU işletim sistemine tuzak (trap) üretir: sayfa hatası.

Bir Sayfa Hatasının Adım Adım İşlenmesi



Altı adım

- **1 Başvuru:** süreç M sayfasına dokunur; biti i'dir.
- **2 Tuzak:** MMU sayfa hatası üretir; işletim sistemi adresin yasal olup olmadığına bakar (değilse: çökme).
- **3 Bulma:** işletim sistemi M sayfasını diskte ve boş bir çerçeve bulur.
- **4 Okuma:** disk → çerçeve (milisaniyeler; bu arada başka bir süreç çalışır).
- **5 Güncelleme:** sayfa tablosu girdisi = çerçeve numarası, bit = v.
- **6 Yeniden başlatma:** aynı komut yeniden çalışır ve bu kez başarılı olur.

BAŞARIM

Sayfa Hatalarıyla Etkin Erişim Süresi

p sayfa hatası oranı olsun ($0 \le p \le 1$): tek bir bellek erişiminin sayfa hatasına yol açma olasılığı.

Formül

- **$EAT = (1 - p) \times t_m + p \times t_fault$**
- **t_m:** bellek erişim süresi (burada 200 ns).
- **t_fault:** sayfa hatası hizmet süresi: tuzak, disk okuması, yeniden başlatma (burada 8 ms; süreyi disk belirler).
- 6. haftadaki TLB formülü gibi ağırlıklı bir ortalama.

Ders: sayfa hatası oranı çok küçük olmalıdır. Onu küçük tutan şey yerelliktir.

```
tₘ = 200 ns,  t_fault = 8 ms = 8 000 000 ns

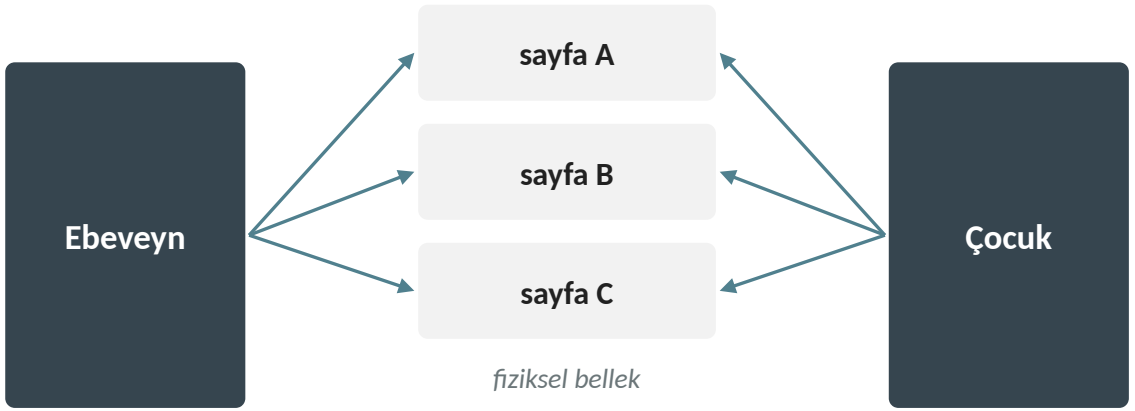
# Örnek: p = 0.001 (1000 erişimde 1 hata)
EAT = 0.999 × 200 + 0.001 × 8 000 000
    = 199.8 + 8 000 = 8 199.8 ns ≈ 8.2 µs
→ RAM'den 41 kat yavaş!

# Yavaşlama %10'un altında  ⇔  EAT < 220 ns
200 + p × (8 000 000 − 200) < 220
p < 20 / 7 999 800 ≈ 2.5 × 10⁻⁶
→ en fazla ≈ 400 000 erişimde 1 hata
```

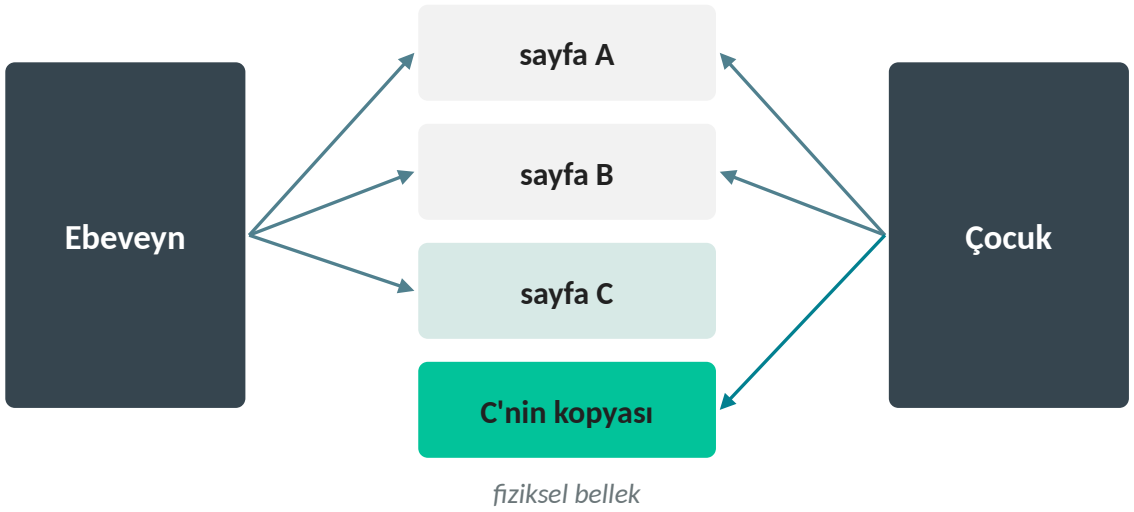
Yazınca Kopyala: fork() Çağrısını Ucuzlatmak

3. hafta: fork(), ebeveyninin kopyası olan bir çocuk oluşturur. Her sayfayı kopyalamak yavaş olurdu ve çoğu zaman boşa giderdi.

1. fork()'tan hemen sonra: sayfalar ortak, salt okunur



2. Çocuk C'ye yazar: yalnızca C kopyalanır



fork()'tan sonra iki süreç tüm sayfaları salt okunur olarak paylaşır. Bir sayfaya ilk yazma bir sayfa hatasına yol açar; işletim sistemi yalnızca o sayfayı kopyalar. Hiç yazılmayan sayfalar (çoğu zaman sayfaların çoğu, özellikle çocuk hemen exec() çağırırsa) hiç kopyalanmaz.

UYGULAMA

Örnek 1: İsteğe Bağlı Sayfalamayı İzlemek

```
static long minor_faults(void)          /* çekirdeğin sayacı */
{
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(void)
{
    long page = sysconf(_SC_PAGESIZE);  /* 4096 bayt    */
    long size = 64 * MB;                /* 16384 sayfa  */

    long f0 = minor_faults();
    char *a = malloc(size);             /* yalnızca sanal */
    long f1 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] = 1;                       /* ilk dokunuş  */
    long f2 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] += 1;                      /* artık bellekte */
    long f3 = minor_faults();
    /* f1-f0, f2-f1, f3-f2 yazdırılır */
}
/* programın tamamı: code/page_faults.c */
```

Neye dikkat edelim?

- malloc() yalnızca sanal alan ayırır: neredeyse hiç hata yok.
- Her sayfaya ilk yazma bir kez hata verir: 16.383 + 1 = 16.384 sayfa.
- İkinci geçiş: 0 hata; sayfalar artık bellekte.

```
$ ./page_faults
page size 4096 bytes; array 64 MB = 16384 pages
after malloc       :      1 new faults
first touch (write):  16383 new faults
second touch       :      0 new faults
```

UYGULAMA

Örnek 2: Yazınca Kopyala, Ölçülmüş Hâliyle

```
char *buf = malloc(SIZE);              /* 32 MB = 8192 sayfa */
memset(buf, 'A', SIZE);                /* sayfalar ebeveynin */

pid_t pid = fork();
if (pid == 0) {                        /* ---- çocuk ----    */
    long f0 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        sum += buf[i];                 /* OKU: paylaşılan    */
    long f1 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        buf[i] = 'B';                  /* YAZ: sayfa kopyası */
    long f2 = minor_faults();
    /* f1 - f0 ve f2 - f1 yazdırılır */
    exit(0);
}
wait(NULL);
printf("parent still sees '%c'\n", buf[0]);
/* programın tamamı: code/cow_fork.c */
```

Neye dikkat edelim?

- Paylaşılan sayfaları okumak: hiç hata yok.
- Yazmak: sayfa başına bir hata (8192); her sayfa ilk yazmada kopyalanır.
- Ebeveynin verisine dokunulmaz.

```
$ ./cow_fork
buffer: 8192 pages of 4096 bytes
child read  all pages:      0 faults
child write all pages:   8192 faults
                       (copy-on-write)
parent still sees  'A'
```

Boş Çerçeve Yok mu? Biri Çıkmak Zorunda

Bellek bilerek fazla dağıtılır. Bir sayfa hatası tüm çerçeveleri dolu bulursa işletim sistemi bir kurban sayfa seçmelidir.

1. Sayfayı bul

İstenen sayfanın diskteki yerini bul.

→

2. Kurban seç

Boş çerçeve yok → bir değiştirme algoritması birini seçer.

→

3. Diske yaz?

Yalnızca kurban kirliyse (değiştirilmişse).

→

4. İçeri oku

İstenen sayfayı yükle; iki sayfa tablosunu güncelle.

→

5. Sürdür

Hata veren süreci kaldığı yerden sürdür.

Değişiklik (kirli) biti

Bir sayfaya yazıldığında donanım bu biti kurar. Temiz bir kurbanın (ör. program kodu) diskte aynısı zaten vardır, bu yüzden atılabilir: iki yerine tek disk aktarımı.

Algoritmaları nasıl karşılaştırırız?

Her birini aynı başvuru dizisi (reference string: programın dokunduğu sayfa numaraları dizisi) ve aynı çerçeve sayısıyla çalıştırıp sayfa hatalarını sayarız. Az olan iyidir.

FIFO: En Eski Sayfayı Değiştir

İlk giren ilk çıkar (First-In, First-Out): kurban, bellekte en uzun süredir duran sayfadır. Bir kuyrukla kolayca gerçeklenir.

Başvuru dizisi: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 çerçeve

	başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
	çerçeve 0	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7
	çerçeve 1		**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0
	çerçeve 2			**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**
	hata?	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

Not

FIFO bir sayfanın ne sıklıkla kullanıldığına bakmaz: 7. başvuruda 0 sayfasını atar, oysa o hemen yine gerekir (11. başvuru). Nane = yeni yüklenen sayfa; soluk sütunlar = hatalar.

15

FIFO ile sayfa hatası

OPT: Gelecekte En Geç Kullanılacak Sayfayı Değiştir

En iyi (optimal; Belady, 1966): kurban, en uzun süre kullanılmayacak olan sayfadır. Olası en düşük hata sayısını verir.

Başvuru dizisi: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 çerçeve

başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
çerçeve 0	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
çerçeve 1		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
çerçeve 2			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
hata?	F	F	F	F		F		F			F			F				F		

Sorun

OPT gelecekteki başvuru dizisini bilmeyi gerektirir; bu yüzden hiçbir gerçek işletim sistemi onu gerçekleyemez. Bir ölçüttür: "algoritmamız en iyinin %10 yakınında".

9

OPT ile sayfa hatası (en az)

LRU: En Uzun Süredir Kullanılmayan Sayfayı Değiştir

En uzun süredir kullanılmayan (Least Recently Used): kurban, en uzun süredir kullanılmamış sayfadır. Geçmişi geleceğin tahmini olarak kullanır.

Başvuru dizisi: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 çerçeve

başvuru	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
çerçeve 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
çerçeve 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
çerçeve 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
hata?	F	F	F	F		F		F	F	F	F			F		F		F		

Gerçekleme

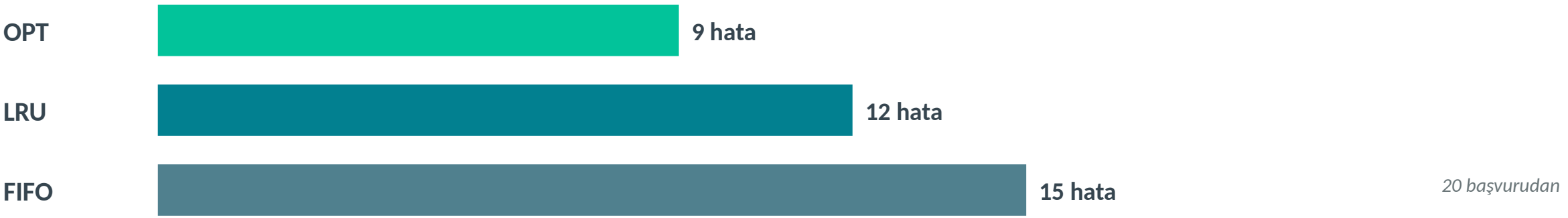
Tam LRU, her bellek erişiminde güncellenen bir zaman damgası ya da bir yığıt (stack) gerektirir; donanımda bu çok pahalıdır. Gerçek sistemler ona yaklaşır (saat algoritması, birazdan).

12

LRU ile sayfa hatası

Üç Algoritma, Tek Başvuru Dizisi

Algoritma	Kurban şu sayfadır…	Hata (3 çerçeve)	Gerçeklenebilir mi?	Belady anomalisi?
FIFO	en uzun süre önce yüklenen	15	evet, çok ucuz	olabilir
OPT	gelecekte en geç kullanılacak	9	hayır (geleceği ister)	asla
LRU	en uzun süredir kullanılmayan	12	yaklaşık olarak	asla



Bu dizide sıralama: OPT (9) < LRU (12) < FIFO (15). Her algoritma 7, 0 ve 1'in ilk kullanımı için 3 zorunlu hatayı öder.

SÜRPRİZ

Belady Anomalisi: Daha Çok Çerçeve, Daha Çok Hata?

FIFO, dizi: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

3 çerçeve → 9 hata

başvuru	1	2	3	4	1	2	5	1	2	3	4	5
çerçeve 0	**1**	1	1	**4**	4	4	**5**	5	5	5	5	5
çerçeve 1		**2**	2	2	**1**	1	1	1	1	**3**	3	3
çerçeve 2			**3**	3	3	**2**	2	2	2	2	**4**	4
hata?	F	F	F	F	F	F	F			F	F	

4 çerçeve → 10 hata

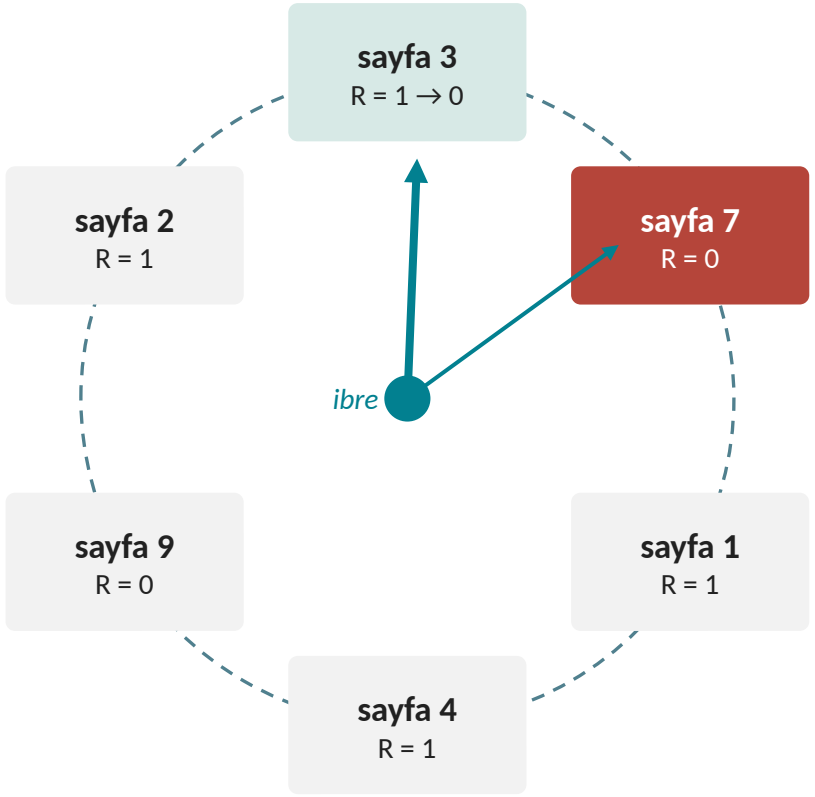
başvuru	1	2	3	4	1	2	5	1	2	3	4	5
çerçeve 0	**1**	1	1	1	1	1	**5**	5	5	5	**4**	4
çerçeve 1		**2**	2	2	2	2	2	**1**	1	1	1	**5**
çerçeve 2			**3**	3	3	3	3	3	**2**	2	2	2
çerçeve 3				**4**	4	4	4	4	4	**3**	3	3
hata?	F	F	F	F			F	F	F	F	F	F

Ne anlama gelir?

- Sağduyu: daha çok bellek asla zarar vermemeli.
- FIFO'da verebilir: 3 çerçeveyle 9, 4 çerçeveyle 10 hata.
- OPT ve LRU asla böyle davranmaz (bunlar "yığıt algoritmaları"dır).
- Aynı dizide LRU 10 → 8 hata verir.

Başvuru Biti ve İkinci Şans (Saat) Algoritması

Bir sayfa her kullanıldığında donanım onun başvuru bitini (R) 1 yapar. İşletim sistemi bu biti okuyabilir ve sıfırlayabilir.

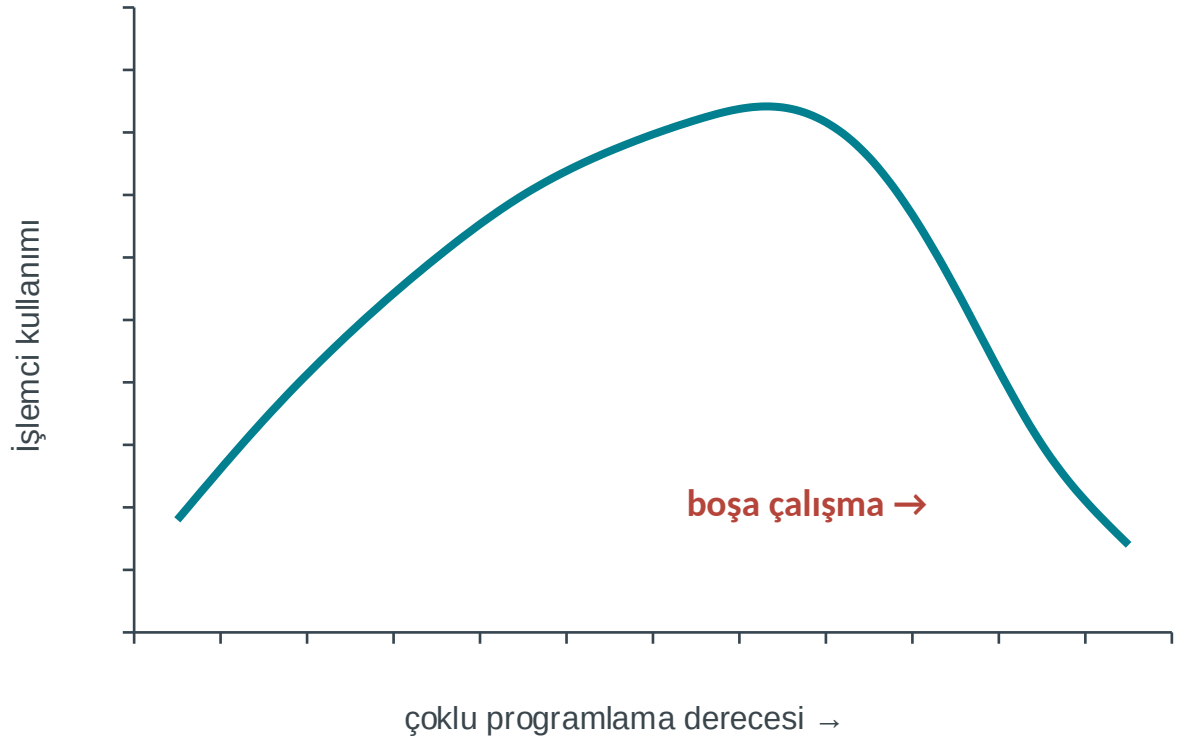


Algoritma

- Çerçeveler bir çember oluşturur; bir "saat ibresi" sıradaki adayı gösterir.
- **R = 1:** ikinci bir şans ver: R'yi 0 yap, ilerle.
- **R = 0:** yakın zamanda kullanılmamış → kurban budur.
- **Örnek:** ibre 3. sayfada (R = 1): biti sıfırla, 7. sayfaya ilerle (R = 0): 7. sayfayı değiştir.
- Bütün R'ler 1 ise ibre tam bir tur atar: FIFO.

Boşa Çalışma: Yoğun Sayfalama, Gerçek İş Yok

Bir sürecin çerçeve sayısı etkin olarak kullandığı sayfalardan azsa, süreç tekrar tekrar hata verir.



kavramsal eğri, ölçülmüş veri değildir

Kısır döngü

- Çok fazla süreç → her birine çok az çerçeve → çok sayıda sayfa hatası.
- Süreçler diski bekler → işlemci kullanımı düşer.
- Saf bir zamanlayıcı boşta duran işlemciyi görür ve daha çok süreç kabul eder…
- …bu da durumu kötüleştirir. Sistem zamanını hesaplamaya değil, sayfalamaya harcar.
- **Çare:** çoklu programlama derecesini düşürmek (süreçleri askıya almak) ya da RAM eklemek.

Çalışma Kümesi Modeli

Çalışma kümesi (working set) WS(t, Δ) = son Δ başvuruda kullanılan farklı sayfaların kümesi. Programın yerelliğini izler.



İşletim sisteminin kuralı

- Her sürece en az |WS| çerçeve ver (çalışma kümesi boyutu).
- Toplam talep D = Σ $|WS_i|$. D > çerçeve sayısı m ise boşa çalışma yaklaşıyor → bir süreci askıya al.

Δ'yı seçmek

- Δ çok küçük: yerelliğin bir kısmını kaçırır.
- Δ çok büyük: birkaç yerelliği kapsar, çerçeve israf eder.
- Daha basit akrabası: sayfa hatası sıklığını izle ve çok sık hata veren süreçlere çerçeve ver.

ARA

10 Dakika Mola

Sırada: dosya sistemleri; dosyalar, erişim yöntemleri, dizinler ve Linux bağlantıları.

BÖLÜM 2

Dosya Sistemleri

İşletim sisteminin en görünür parçası.

Bir Dosya ve Öznitelikleri

Dosya, ikincil bellekte saklanan, adı olan bir ilişkili bilgiler topluluğudur; kullanıcının kaydedebileceği en küçük birimdir.

Öznitelik	Anlamı	Örnek (Linux, notes.txt)
Ad	İnsanın okuyabileceği ad; Linux'ta diğer özniteliklerle değil, dizinde tutulur	notes.txt
Tanımlayıcı	Dosya sistemi içinde benzersiz bir numara	düğüm (inode) 368662
Tür	Normal dosya, dizin, bağlantı, aygıt, ...	normal dosya
Konum	Veri bloklarının aygıttaki yeri	düğümdeki blok işaretçileri
Boyut	Bayt (ya da blok) cinsinden güncel boyut	6 bayt (512 B'lık 8 blok)
Koruma	Kim okuyabilir, yazabilir, çalıştırabilir	-rw-rw-r-- sahibi student
Zaman damgaları	Oluşturma, son erişim, son değişiklik	Access / Modify / Change

stat notes.txt çalıştırırsanız bu tablonun neredeyse her satırını görürsünüz. İşletim sistemi bu öznitelikleri bir dosya denetim bloğunda (Linux'ta: düğüm, inode) tutar.

Bir Dosya Üzerinde Sekiz Temel İşlem

create (oluştur)

Yer bul; dizine bir girdi ekle. open(..., O_CREAT)

open (aç)

Adı bir kez ara; dosya tanımlayıcısı al. open()

read (oku)

Geçerli konumdan baytları kopyala. read()

write (yaz)

Geçerli konuma baytları kopyala. write()

seek (konumlan)

Geçerli konumu taşı; G/Ç yok. lseek()

close (kapat)

Açık dosya tablosu girdisini bırak. close()

delete (sil)

Adı kaldır; kullanılmıyorsa yeri boşalt. unlink()

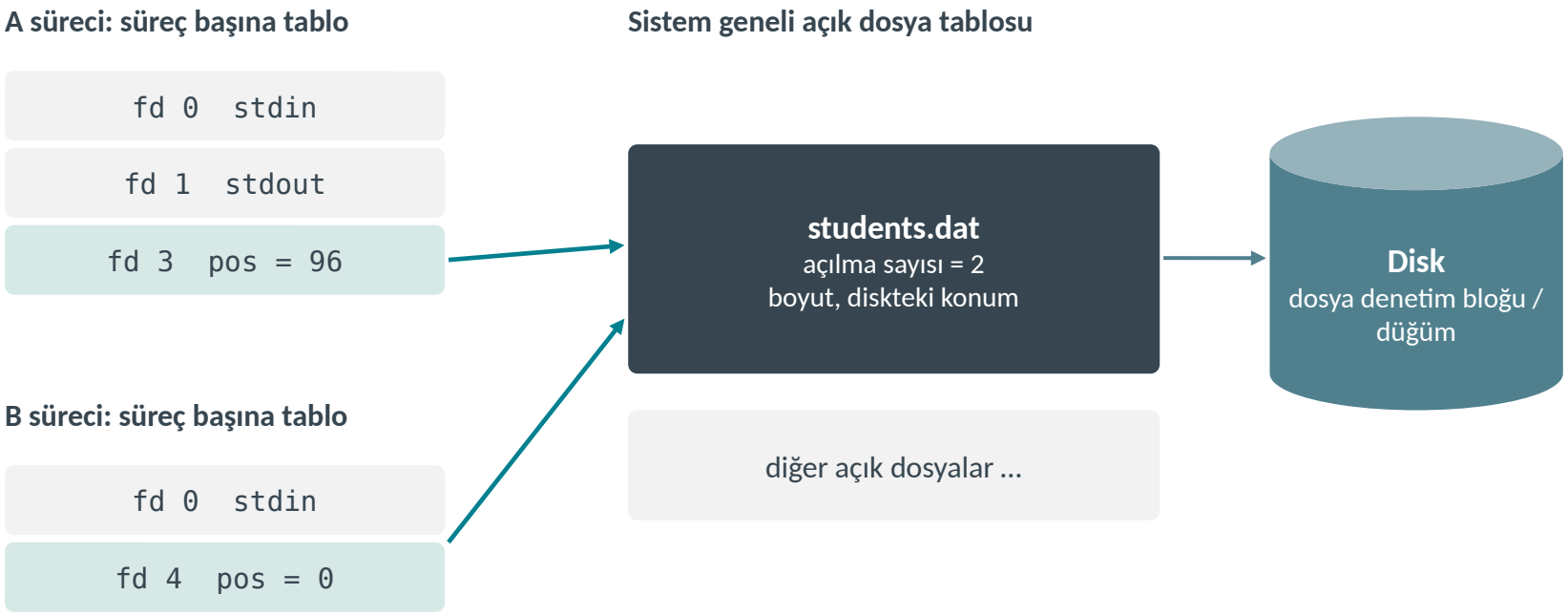
truncate (kırp)

İçeriği sil, öznitelikleri koru. truncate()

Bir kez açın, sonra her okuma/yazma için küçük tamsayıyı (dosya tanımlayıcısı) kullanın: işletim sistemi dizini yeniden aramak zorunda kalmaz. 0, 1, 2 zaten açıktır: stdin, stdout, stderr (2. hafta).

Açık Dosya Tabloları

open() yavaş işi (dizin araması, izin denetimi) bir kez yapar ve sonucu tablolarda saklar.



Kim neyi tutar?

- **Süreç başına:** dosya işaretçisi (geçerli konum) ve erişim kipi.
- **Sistem geneli:** dosyanın tüm kullanıcılarınca paylaşılan bilgiler ve bir açılma sayısı.
- close() sayıyı bir azaltır; 0 olunca girdi silinir.

Dosya Türleri ve Uzantılar

Dosya türü	Olağan uzantılar	İçerik
Kaynak kod	.c .py .java	program metni
Nesne / çalıştırılabilir	.o .exe (Linux'ta yok)	makine kodu
Betik	.sh .bat	kabuk için komutlar
Metin / belge	.txt .pdf .docx	metin, biçimli belgeler
Çoklu ortam	.jpg .mp3 .mp4	görüntü, ses, video
Arşiv	.zip .tar.gz	sıkıştırılmış birden çok dosya

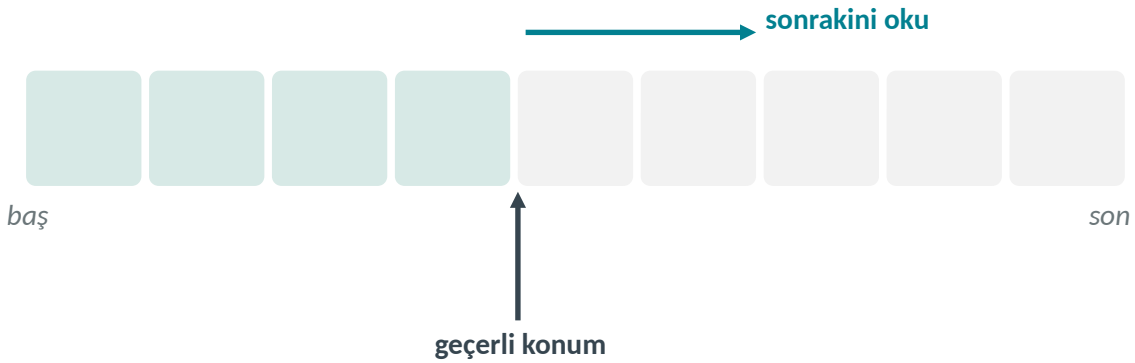
```
$ file file_access.c
file_access.c: C source, ASCII text
$ file file_access
file_access: ELF 64-bit LSB pie executable,…
$ file photo.txt
photo.txt: JPEG image data, JFIF standard 1…
$ file students.dat
students.dat: data
```

Ad ve içerik

- Windows hangi programın açacağını uzantıya göre seçer.
- Linux'ta uzantı yalnızca bir ipucudur; `file` içeriği okur ("sihirli sayı"). photo.txt aslında bir JPEG'dir.

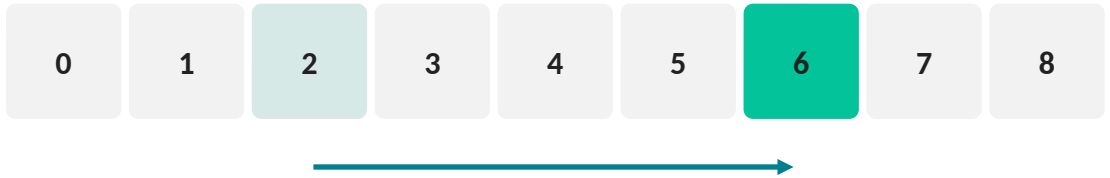
Sıralı Erişim ve Doğrudan Erişim

Sıralı: sonrakini oku / sonrakini yaz / başa sar



- Bilgi sırayla, bir kayıt ötekinin ardından işlenir.
- Teybin modeli; çoğu programın da modeli: derleyiciler, düzenleyiciler, medya oynatıcılar.
- C'de: art arda read() çağrıları; konum kendiliğinden ilerler.

Doğrudan (göreli): n'yi oku / n'ye yaz / n'ye git



2. bloktan 6. bloğa atla; aradakiler okunmaz

- Dosya, sabit boyutlu blok ya da kayıtlardan oluşan numaralı bir dizidir.
- Herhangi bir kayıt herhangi bir sırada okunabilir: diskin modeli.
- Veritabanları, uçak rezervasyonları. C'de: önce lseek(), sonra read().

Çözümlü Örnek: 1000. Kayıt Nerede?

Soru

- Bir dosya, 0'dan numaralanmış, her biri 100 bayt olan öğrenci kayıtları tutuyor.
- Dosya sistemi 4096 baytlık bloklar kullanıyor.
- 1000. kayıt hangi blokta ve o bloğun neresinde başlıyor?

```
bayt konumu = kayıt no × kayıt boyutu
            = 1000 × 100 = 100 000

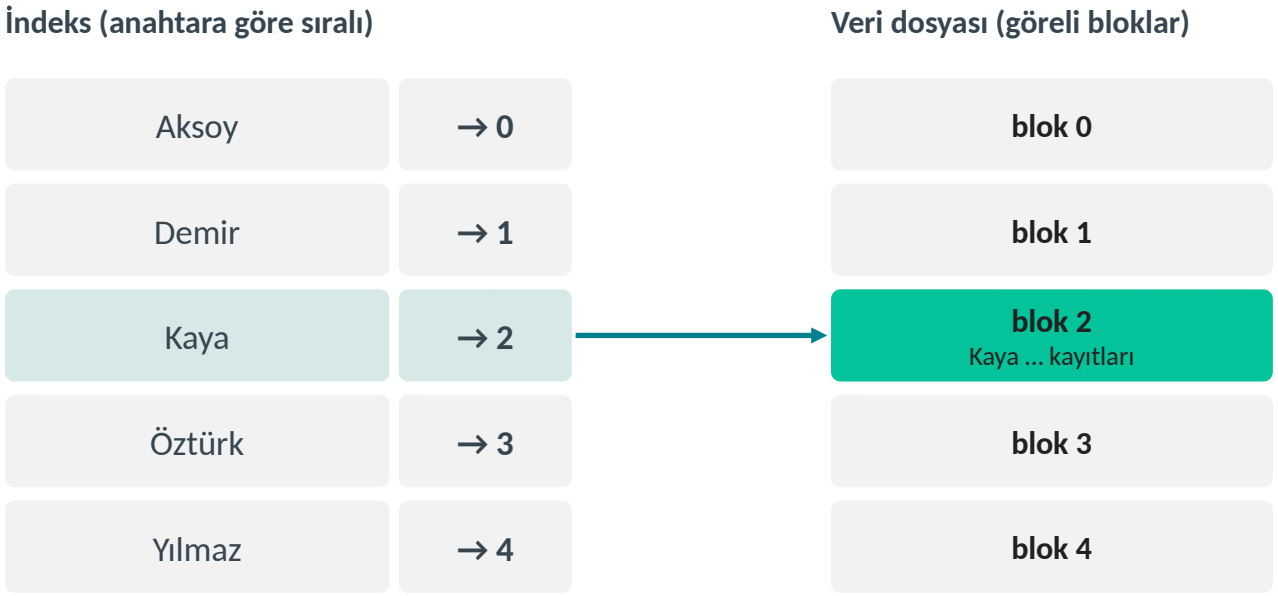
blok        = ⌊100 000 / 4096⌋ = 24
blok içi    = 100 000 − 24 × 4096
            = 100 000 − 98 304 = 1 696

son bayt    = 100 099  → yine 24. blok
```

Yani: 24 numaralı göreli bloğu oku ve 1696 … 1795 arasındaki baytları al. C'de bu tek satırdır: lseek(fd, 1000 * 100, SEEK_SET); ardından read(fd, rec, 100). İşletim sistemi bayt konumunu tam yukarıdaki gibi blok numarasına çevirir. Gösterim programımız 32 baytlık kayıtlar kullanır: 3. kayıt 3 × 32 = 96. baytta başlar.

İndeksli Erişim: Önce Bak, Sonra Atla

Küçük bir indeks dosyası anahtarları blok numaralarına eşler; indekste ara, sonra tek bir bloğu doğrudan oku.

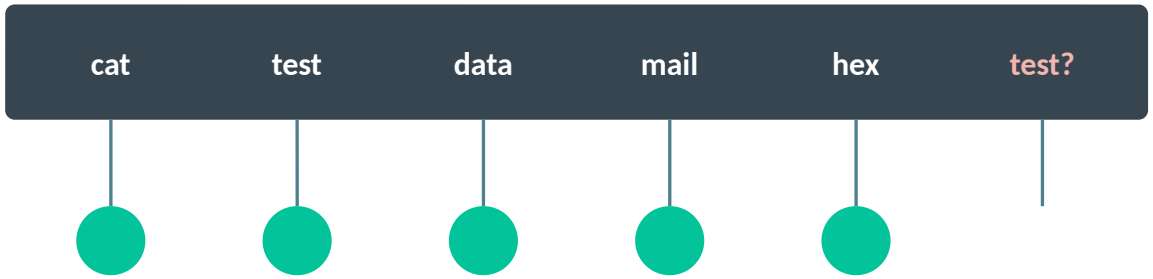


Neden ve nasıl?

- Tüm dosya yerine küçük, sıralı bir indekste ara (hızlı).
- Sonra doğru bloğu tek bir doğrudan okumayla getir.
- Çok büyük dosyalar: indeksin indeksi (çok düzeyli).
- Veritabanlarının (B-ağaçları) ve dosya sistemlerinin bir dosyanın kendi bloklarını bulma yönteminin temeli.

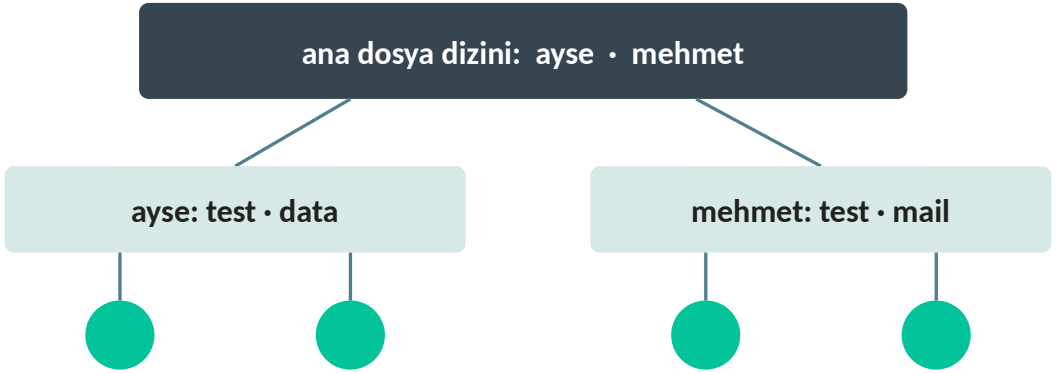
Tek Düzeyli ve İki Düzeyli Dizinler

Tek düzeyli: herkese tek bir dizin



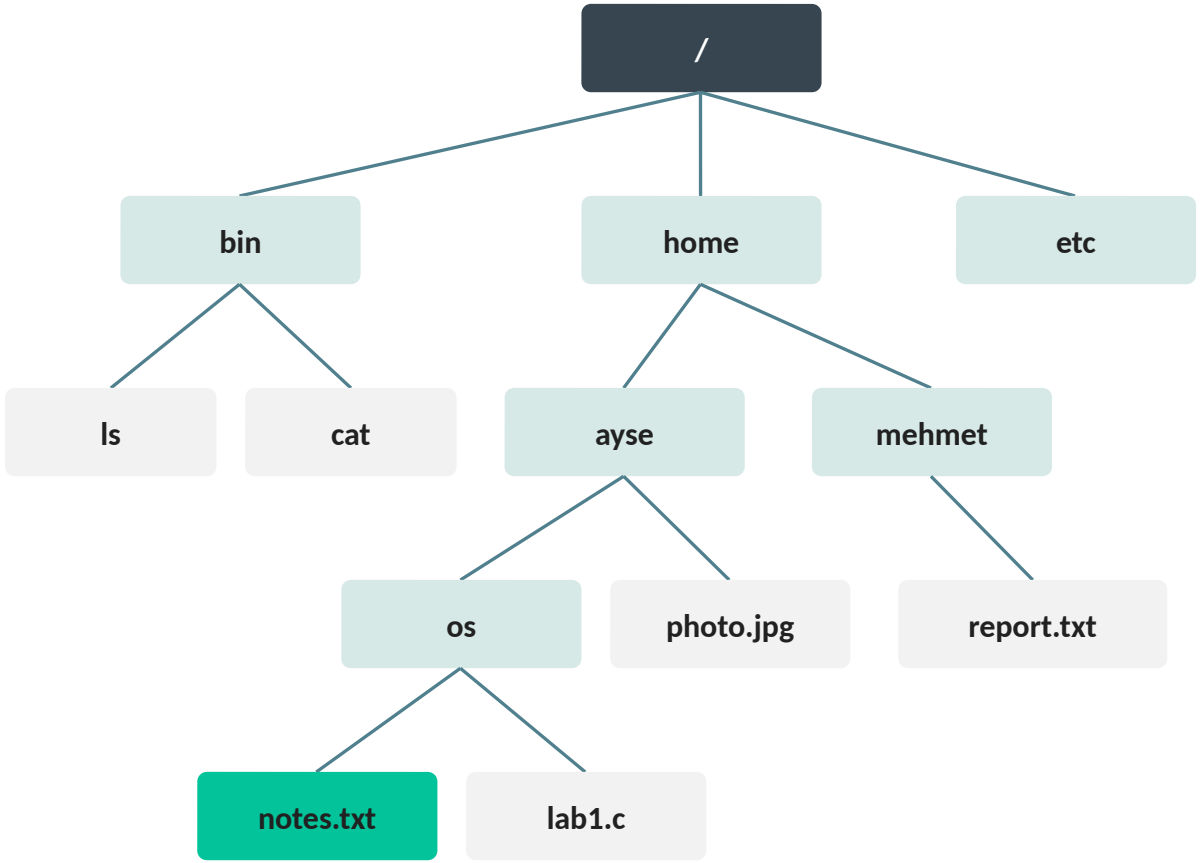
- Basit, ama her dosya adı tüm kullanıcılar arasında benzersiz olmalı.
- İki kullanıcı da bir dosyasına test adını veremez.
- Tek listede yüzlerce dosya: düzenlemek imkânsız.

İki düzeyli: her kullanıcıya bir dizin



- Her kullanıcının özel bir dizini var: ikisinin de test dosyası olabilir.
- Yol adı = kullanıcı + dosya adı, ör. /ayse/test.
- Ama alt klasör yok ve kullanıcılar arasında dosya paylaşmak zahmetli.

Ağaç Yapılı Dizinler ve Yol Adları



soluk = dizin, beyaz = dosya

Bir dosyayı adlandırmanın iki yolu

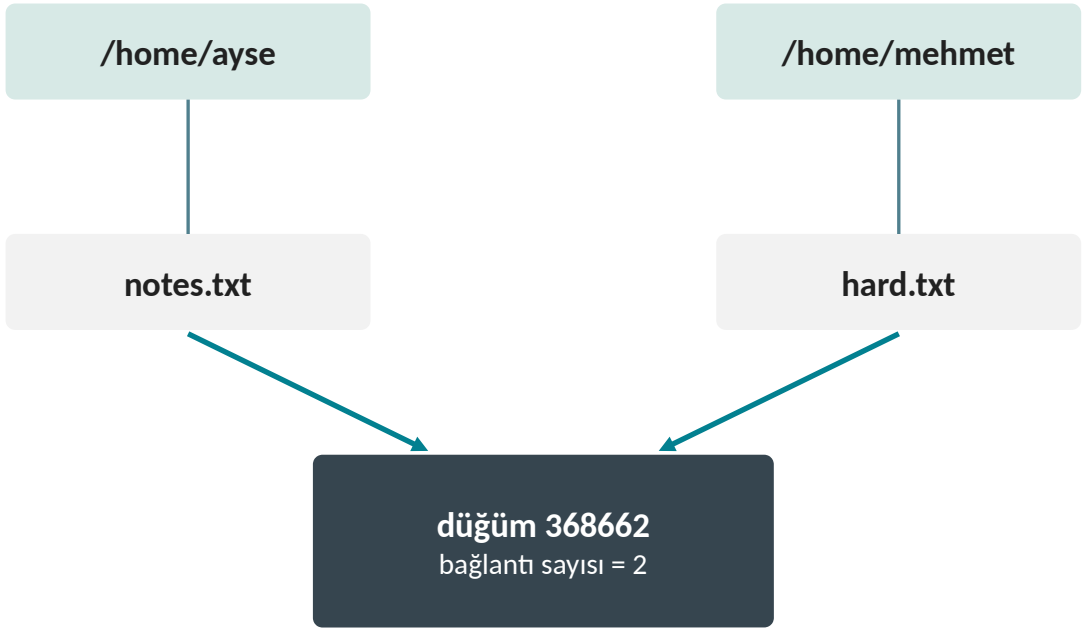
- **Mutlak yol:** kök dizinden (/) başlar.
- **Göreli yol:** geçerli dizinden başlar.
- **. ve ..** bu dizin ve üst dizin demektir.
- 9. hafta: pwd, cd ve ls ile bu ağaçta dolaşacaksınız.

```
# mutlak: her yerden çalışır
/home/ayse/os/notes.txt

# göreli, geçerli dizin /home/ayse
os/notes.txt

# göreli, geçerli dizin /home/ayse/os
../../mehmet/report.txt
```

Döngüsüz Çizgeler: Bağlantılarla Paylaşım



Döngüsüz çizge (acyclic graph)

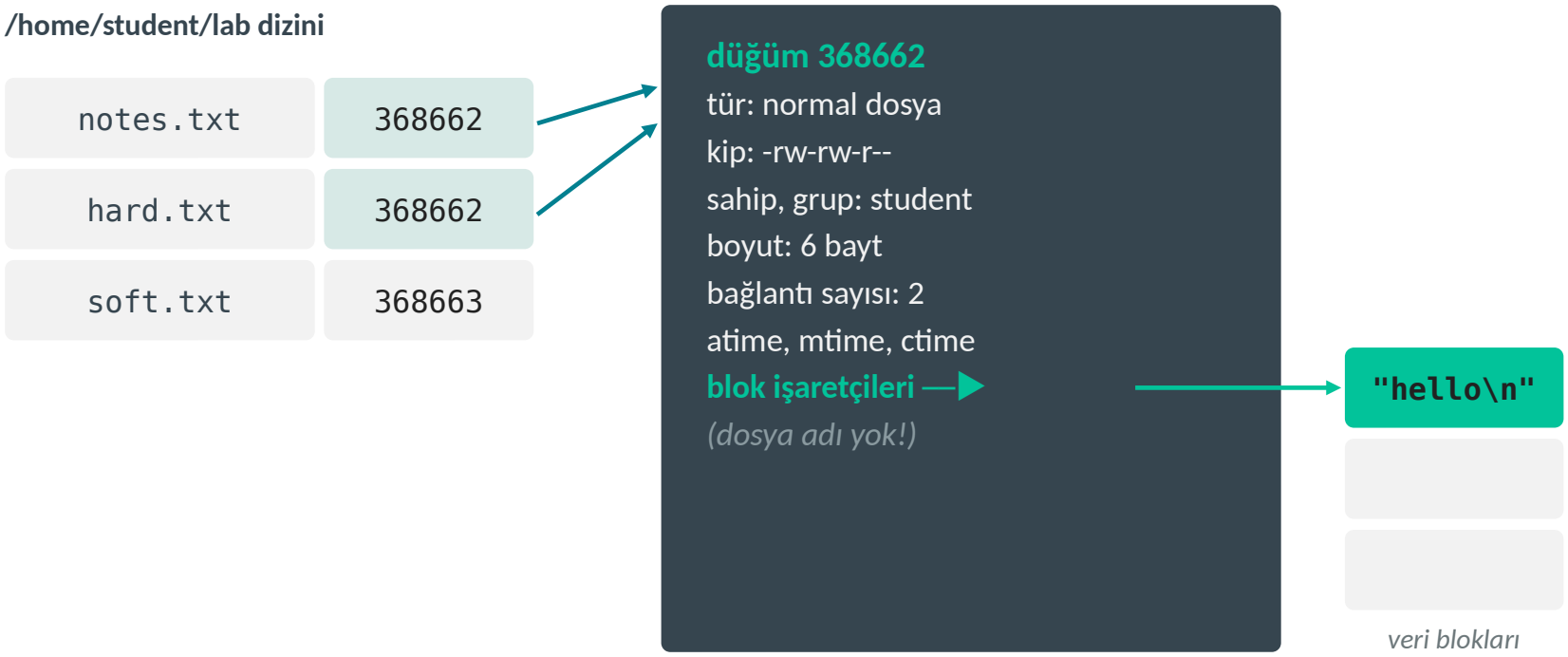
- Bir dosya (ya da dizin) birden çok dizinde yer alabilir: kopyalanmaz, paylaşılır.
- **Sabit bağlantı (hard link):** aynı düğüm için ikinci bir dizin girdisi. Bir adı silmek dosyayı silmez.
- **Sembolik bağlantı:** bir yol tutan küçük bir dosya; hedef silinirse boşta (sarkık) kalır.

Genel çizge (döngülere izin veren)

- Bir üst dizine geri dönen bağlantı döngü oluşturur: aramalar sonsuza dek dönebilir, silinmiş bir alt ağaç hiç "kullanılmaz" duruma gelmeyebilir.
- Linux dizinlere sabit bağlantıyı yasaklar; sembolik bağlantı döngüleri bir hatayla (ELOOP) durur.

Linux Ne Gösterir: Dizin Girdileri ve Düğümler

Dizin, (ad, düğüm numarası) çiftlerinden oluşan bir tablodur. Bir dosyayla ilgili diğer her şey onun düğümünde (inode) durur.



Kendiniz görün

- ls -i: düğüm numaraları.
- ls -l: 2. sütun = bağlantı sayısı.
- stat dosya: düğümün tamamı.
- df -i: boş düğümler.

UYGULAMA

Örnek 3: ls -li, ln, ln -s ve stat

```
$ echo "hello" > notes.txt
$ ln notes.txt hard.txt
$ ln -s notes.txt soft.txt
$ ls -li
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 hard.txt
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 notes.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 368662      Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1002/ student)
Modify: 2026-10-08 12:20:48.530146698 +0000
Change: 2026-10-08 12:20:48.534146698 +0000
$ rm notes.txt
$ cat hard.txt
hello
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -li
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 hard.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
```

Neye dikkat edelim?

- notes.txt ve hard.txt: aynı düğüm, bağlantı sayısı 2.
- soft.txt: kendi düğümü var; boyutu 9 = "notes.txt" dizgesinin uzunluğu.
- Değişim zamanı (ctime) ilerledi: ln düğümü güncelledi.
- rm sonrasında: hard.txt hâlâ çalışır, soft.txt boşta kalır.

UYGULAMA

Örnek 4: C'de Sıralı ve Doğrudan Erişim

```
#define RECSZ 32                        /* kayıt başına bayt */
int fd = open("students.dat", O_CREAT | O_RDWR | O_TRUNC, 0644);
/* ... 5 kayıt yazılır: "0 Ayse", "1 Mehmet", ... "4 Zeynep" */

lseek(fd, 0, SEEK_SET);                 /* 1) sıralı        */
while (read(fd, rec, RECSZ) == RECSZ)
    printf("  [%s]", rec);

lseek(fd, 3 * RECSZ, SEEK_SET);         /* 2) doğrudan: bayt 96 */
read(fd, rec, RECSZ);

lseek(fd, 1 * RECSZ, SEEK_SET);         /* 3) 1. kaydın üzerine */
write(fd, rec, RECSZ);
close(fd);
/* programın tamamı: code/file_access.c */
```

Ne oluyor?

- read() dosya işaretçisini her seferinde 32 bayt ilerletir: sıralı.
- lseek() işaretçiyi herhangi bir yere koyar; sonraki read() ya da write() oradan başlar: doğrudan.
- 1. kayda yazmak diğer kayıtlara dokunmaz.

```
$ ./file_access
sequential:  [0 Ayse]  [1 Mehmet]  [2 Elif]  [3 Can]  [4 Zeynep]
direct: record 3 at byte 96 -> [3 Can]
after update: record 1 -> [1 Mehmet (updated)]
file size: 160 bytes
```

KISA SINAV

Kendinizi Sınayın

1. Başvuru dizisi 1, 2, 3, 1, 4, 1, 2 ve 3 çerçeve: FIFO ile kaç sayfa hatası olur? LRU ile?

2. t_m = 200 ns, t_fault = 8 ms, p = 0,0001. Etkin erişim süresi nedir?

3. Kayıtlar 64 bayt, bloklar 4096 bayt. 200. kayıt hangi blokta, hangi konumda başlar?

4. ln a.txt b.txt ve rm a.txt komutlarından sonra veriyi hâlâ okuyabilir misiniz? b.txt ln -s ile yapılmış olsaydı?

7. Hafta: Akılda Kalması Gerekenler

- **Sanal bellek:** süreç büyük bir adres alanı görür; yalnızca kullanılan sayfalar RAM'dedir. İsteğe bağlı sayfalama bir sayfayı ilk kullanımda yükler.
- **Sayfa hatası:** geçersiz bir sayfaya erişim işletim sistemine tuzak üretir; o da sayfayı yükleyip komutu yeniden başlatır. EAT = $(1 - p)\cdot t_m + p\cdot t_fault$, yani p çok küçük olmalı.
- **Yazınca kopyala:** fork() sayfaları taraflardan biri yazana kadar paylaştırır.
- **Değiştirme:** FIFO (basit, Belady anomalisi), OPT (en iyi, geleceği ister), LRU (iyi, saat algoritmasıyla yaklaşılır). Örnek: 15 / 9 / 12 hata.
- **Boşa çalışma:** çok az çerçeve → sürekli sayfalama; her çalışma kümesini bellekte tut.
- **Dosyalar:** öznitelikleri olan adlandırılmış veri (düğümde); açık dosya tabloları; sıralı, doğrudan, indeksli erişim.
- **Dizinler:** tek düzeyli → iki düzeyli → ağaç → döngüsüz çizge (sabit ve sembolik bağlantılar).

Vize Kontrol Listesi: 1–7. Haftalar

H1 Tarihçe

Toplu işleme, çoklu programlama, zaman paylaşımı; boşta duran işlemci tasarımı nasıl yönlendirdi; UNIX ve Linux.

H2 Temeller

Tanımlar, hedefler, kesmeler, bellek hiyerarşisi, ikili mod, sistem çağrıları, yapılar.

H3 Süreçler

Hizmetler, sistem çağrısı türleri, süreç durumları, PCB, bağlam değişimi, fork().

H4 İş parçacıkları

Kullanıcı ve çekirdek iş parçacıkları, çoklu iş parçacığı modelleri, zamanlama ölçütleri.

H5 Eşzamanlama

Gantt şemalarıyla FCFS, SJF/SRTF, Öncelik, RR; yarış durumu, mutex, semafor.

H6 Bellek

Mantıksal ve fiziksel adres, MMU, parçalanma, sayfalama, TLB ve EAT, bölümleme.

H7 SB ve dosyalar

Sayfa hataları, p ile EAT, FIFO/OPT/LRU tabloları, boşa çalışma; erişim yöntemleri, dizinler, bağlantılar.

Nasıl hazırlanmalı?

Slaytlardaki her çözümlü örneği ve sınavı, notlardaki tekrar sorularını yeniden çözün.

GELECEK HAFTA

Vize Sınavı (8. hafta)

8. haftanın konuları:

- 1–7. haftaları kapsar: slaytlar ve ders notları
- Alıştırın: Gantt şemaları, EAT (TLB ve sayfa hataları), FIFO/OPT/LRU tabloları, blok hesabı
- Öğrenci kimliğinizi, kaleminizi ve izin verilirse basit bir hesap makinesi getirin
- Saat ve salon: dersin Telegram kanalını takip edin
- Vizeden sonra 9. hafta uygulamalı Linux ile başlıyor: kabuk ve dizinlerde gezinme

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz